

Tech Nation Visa - Exceptional Promise

Criteria: Mandatory Criteria (Proof of Technical Innovation & Product Architecture)

Applicant: Odeyemi Olusegun Israel

Evidence 1: AMTTP Product Functional Demonstration & System Architecture

1. Live Functional Demonstration

The complexity of the Anti-Money Laundering Transaction Transfer Protocol (AMTTP) spans multiple domains: Machine Learning infrastructure, Web3 decentralized settlement, and dual-client frontend architectures. To best evaluate the scale of this technical achievement, please view the short functional demonstration of the final product in action.

[CLICK HERE TO VIEW THE 3-MINUTE VIDEO DEMONSTRATION]

(Hosted securely via unlisted URL)

2. High-Level Macro Architecture

As the sole Founder and Lead Architect, I engineered the entire data pipeline from initial consumer wallet interaction through to predictive ML risk scoring and final immutable blockchain settlement via Zero-Knowledge Proofs (ZK-SNARKs).

Live Production Service Map:

```
User → Flutter Consumer App (port 3010)
User → Next.js War Room Dashboard (port 3006)
↓
Oracle Gateway (Express.js, port 3001)
↓
Compliance Orchestrator (FastAPI, port 8007)
↓ parallel fan-out via aiohttp
+-- ML Risk Engine           (port 8000) -- XGBoost/LightGBM ensemble
+-- Graph Service            (port 8001) -- Memgraph/GraphSAGE
+-- FCA Policy Engine         (port 8003) -- UK regulatory rules
+-- Sanctions Screening       (port 8004) -- OFAC/EU/UK lists
+-- Monitoring Service        (port 8005) -- Real-time alerting
+-- GeoRisk Service           (port 8006) -- Jurisdiction scoring
+-- Data Integrity            (port 8008) -- Merkle proof validation
+-- Explainability            (port 8009) -- SHAP/LIME explanations
+-- ZK-NAF Verifier           (port 8010) -- Groth16 ZK-SNARK proofs
↓
Solidity Smart Contracts (UUPS Proxy) → EVM Settlement
↓
Infrastructure: MongoDB (27017) | Redis (6379) | Memgraph (7687)
```

Figure 1: Full-stack infrastructure mapping Flutter/Next.js to 10 Python microservices and the Solidity execution layer — 14 independently deployable services engineered by a single architect.

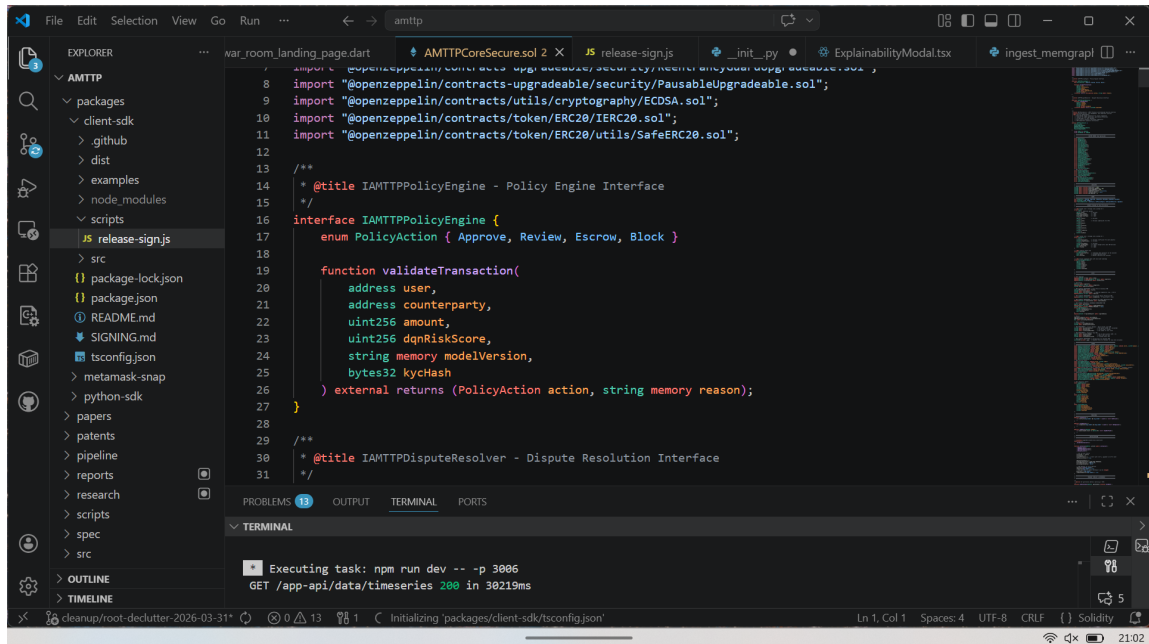


Figure 1b: VS Code workspace showing simultaneous development across Dart (Flutter), Solidity (smart contracts), JavaScript (deploy scripts), Python (ML pipeline), and TypeScript (Next.js) — all authored by a single full-stack engineer. The terminal shows the Next.js War Room dev server running live on port 3006.

3. The Enterprise "War Room": ML Risk Infrastructure

Below is the administrative dashboard (Next.js) directly interfacing with the Python/FastAPI ML inference pipeline. This system processes high-throughput data streams, surfacing anomalies via distilled XGBoost student models.

FastAPI Risk Scoring Endpoint (from `ml/Automation/risk_engine/`):

```
@app.post("/score")
async def score.transaction(request: TransactionRequest):
    features = extract_features(request)
    tabular_score = xgb_model.predict_proba(features)[: ,1]
    graph_score = await memgraph.query_risk(request.from_addr)
    combined = 0.7 * tabular_score + 0.3 * graph_score
    return {"risk_score": combined, "action": get_policy_action(combined)}
```

Figure 2: FastAPI prediction endpoint (port 8000) combining XGBoost tabular scoring with Memgraph graph analysis. The War Room dashboard (Next.js, port 3006) consumes this API to render real-time risk panels and transaction alerts.

4. Consumer Edge & Decentralized Settlement

The final tier involves the cross-platform consumer client (built in Flutter) routing safe transactions to EVM-compatible blockchains. Compliance checks are verified entirely on-chain without leaking private keys or PII.

ZK-SNARK On-Chain Verification (from `contracts/zknaf/sanctions_non_membership_verifier.sol`)

```
function verifyProof(
    uint[2] calldata _pA, uint[2][2] calldata _pB,
    uint[2] calldata _pC, uint[4] calldata _pubSignals
) public view returns (bool) {
    assembly {
        // G1 point multiplication via EVM precompile 7
        function g1_mulAccC(pR, x, y, s) {
            success := staticcall(sub(gas(), 2000), 7, mIn, 96, mIn, 64)
        }
        // Groth16 pairing check via precompile 8
        function checkPairing(pA, pB, pC, pubSignals, pMem) -> isOk {
            g1_mulAccC(_pVk, IC1x, IC1y, calldataload(add(pubSignals, 0)))
            g1_mulAccC(_pVk, IC2x, IC2y, calldataload(add(pubSignals, 32)))
            isOk := and(success, mload(_pPairing))
        }
    }
}
```

Figure 3: Groth16 ZK-SNARK proof verification in Solidity — sanctions non-membership is verified on-chain using BN254 elliptic curve pairings without revealing the user's identity. The Flutter consumer app initiates transactions that ultimately settle through this verification layer.